

Emre Kalem Robot Arm

Arduino Uno wiring, calibration, and programming package for the built arm, the conveyor load/unload arm, and a future teleop/training arm. Generated for bench use on 2026-07-31.

Bottom line

Use the Uno for signal only. Power all servo red wires from the separate regulated 5V 10A supply, tie the servo ground bus to Arduino GND, and leave the automatic conveyor program locked until the physical limits are calibrated.

1. What This Assumes

This guide assumes the MakerWorld Emre Kalem 6-axis robot arm: 1 Arduino Uno or Mega, 4 MG995/MG996R 180-degree servos, 3 MG90S 180-degree servos, a KCD1 rocker switch, one 608 bearing, two 6203 bearings, a 5V 10A power supply, M3 hardware including common 6 mm / 10 mm / 14 mm lengths, and jumper wiring.

The wiring and sketches target an Arduino Uno because that is what you asked for. The arm has 6 logical joints, but 7 physical servos because the shoulder/Arm A joint uses two opposed servos. The code mirrors the second shoulder servo as 180 - logical angle.

Logical joint	Physical servo(s)	Default signal pin(s)	Starting limits
Base	1x MG995/MG996R	D3	10 to 170, home 90
Shoulder / Arm A	2x MG995/MG996R, one mirrored	D4 and D5	45 to 135, home 90
Elbow / Arm B	1x MG995/MG996R	D6	30 to 150, home 90
Wrist pitch	1x MG90S	D9	25 to 155, home 90
Wrist roll	1x MG90S	D10	10 to 170, home 90
Gripper	1x MG90S	D11	35 to 110, home 70

2. The 4-Hour Wake-Up Checklist

- 1 Inspect the printed arm with power off. Each joint must move through its expected travel without binding, rubbing, or twisting servo wires.
- 2 Set the 5V supply to about 5.0V before connecting the servo rail. Cover mains terminals and strain-relieve the output wires.
- 3 Build the servo power bus: power-supply +5V -> fuse or inline protection -> KCD1 rocker switch -> servo red bus. Power-supply 0V -> servo ground bus.
- 4 Connect Arduino GND to the servo ground bus. Do not connect the external 5V servo rail to the Uno 5V pin while the Uno is powered by USB.
- 5 Upload the calibration sketch. Attach only one servo at a time. Center at 90 degrees, install/tighten the horn at mechanical home, then detach before moving to the next channel.
- 6 Upload the main controller sketch. Open Serial Monitor at 115200 baud, type help, then jog with small nudges.
- 7 Only after the motion is proven, fill the conveyor waypoint worksheet and copy those angles into the waypoint array.

3. Safety and Power Rules

No live conveyor experiments yet

Bench-test the arm away from the conveyor first. Hobby servos do not have safety-rated position feedback or torque limiting. Do not let the arm enter a moving conveyor path until the conveyor is locked out or otherwise made safe by a qualified person.

- Treat the KCD1 rocker as a servo power enable, not a safety-rated emergency stop.
- Keep fingers clear of shoulder, elbow, wrist, and gripper pinch points when servo power is on.
- If a servo growls, stalls, or heats up, remove servo power immediately and reduce the software limits.
- The Uno 5V pin and digital I/O pins cannot supply servo motor current. They are for logic and signal, not motor power.
- A 5V 10A supply is enough to start the bench work, but four MG996R-class servos can demand large current spikes. Avoid commanding all high-torque joints to slam at once.

4. Wiring Architecture

Use this as the mental model before touching wires:

```
AC mains -> covered 5V 10A DC supply
          -> fuse/inline protection
          -> KCD1 rocker switch
          -> +5V servo bus
          -> every servo red wire

5V supply 0V/GND -> servo ground bus
                  -> every servo brown/black wire
                  -> Arduino GND

Arduino Uno D3/D4/D5/D6/D9/D10/D11 -> servo signal wires only
Arduino Uno USB -> programming + serial commands
```

Common ground is mandatory

The servo signal is measured relative to ground. If the servo supply ground and Arduino ground are not tied together, the servo may jitter, ignore commands, or behave unpredictably.

Servo lead color	Connects to	Notes
Red	External switched +5V servo bus	Never to an Uno digital pin. Avoid Uno 5V for servo motor power.
Brown or black	Servo ground bus and Arduino GND	This is the shared reference for the signal pulse.
Orange, yellow, or white	Uno signal pin D3/D4/D5/D6/D9/D10/D11	Signal only; current is tiny compared with motor current.

5. Servo-by-Servo Wiring

Physical servo	Joint	Signal	Power	Ground	Calibration note
base	Base rotation	D3	Red to switched 5V	Brown/black to ground bus	Matches Emre original sketch; verify the base cannot hit wiring.
arm_a_left	Shoulder primary	D4	Red to switched 5V	Brown/black to ground bus	Moves with logical Arm A.
arm_a_right	Shoulder mirrored	D5	Red to switched 5V	Brown/black to ground bus	Code sends 180 - Arm A. Flip the mirror flag if wrong.
arm_b	Elbow	D6	Red to switched 5V	Brown/black to ground bus	Keep limits conservative while unloaded.
wrist_a	Wrist pitch	D9	Red to switched 5V	Brown/black to ground bus	Watch for printed wrist binding.
wrist_b	Wrist roll	D10	Red to switched 5V	Brown/black to ground bus	Leave a wire service loop.
gripper	Gripper	D11	Red to switched 5V	Brown/black to ground bus	Find open/closed values gently.

Why this pin map?

D3, D4, D5, D6, D9, D10, and D11 match Emre's original Arduino sketch and keep D0/D1 free for USB serial. A contiguous D2-D8 signal strip can also work, but only if you update the SERVO_PINS array in both included sketches.

6. Arduino Upload and Calibration

Install and upload

- 1 Open Arduino IDE.
- 2 Select Tools -> Board -> Arduino AVR Boards -> Arduino Uno.
- 3 Select the correct COM port.
- 4 Open docs/instructions/emre-kalem-robot-arm/arduino/emre_kalem_arm_calibrate/emre_kalem_arm_calibrate.ino.
- 5 Upload. Open Serial Monitor at 115200 baud.

Calibration serial commands

Command	Action
0..6	Select physical servo channel.
a	Attach selected servo and write its current angle.
d	Detach selected servo.
c	Center selected servo at 90 degrees.
+ / -	Nudge selected servo by 1 degree.
] / [	Nudge selected servo by 5 degrees.
p	Print all servo angles and attach status.

Horn-centering method

With the horn loose or removed, select a servo, attach it, send c to center at 90 degrees, then install the horn in the closest mechanical home position. If the closest tooth is a little off, use the servoTrim array in the main sketch for small corrections.

7. Main Controller Programming

After calibration, upload `emre_kalem_arm_uno_controller.ino`. The main sketch starts in serial manual-control mode. It has a placeholder conveyor program, but playback is locked by default with `ALLOW_DEMO_PROGRAM=false`.

Command	Meaning
help	Show command list.
status	Print logical joint angles and physical servo outputs.
home	Move slowly to configured home pose.
attach	Attach all servos and write current setpoints.
detach	Stop sending servo pulses. The arm may sag.
b+ / b-	Base nudge.
a+ / a-	Shoulder / Arm A nudge.
e+ / e-	Elbow / Arm B nudge.
p+ / p-	Wrist pitch nudge.
r+ / r-	Wrist roll nudge.
g+ / g-	Gripper nudge.
set base 90	Move a named joint to a specific angle.
speed 25	Set milliseconds per degree for manual motion.
step 2	Set nudge size in degrees.
demo	Run conveyor waypoints only after unlocking in code.

First manual motion

Start with step 1 and speed 30 if the arm is near anything solid. Move one joint at a time. If the mirrored shoulder fights itself, cut servo power and flip the matching `SERVO_MIRRORED` value or adjust horn orientation.

8. Conveyor Load/Unload Arm Plan

The second arm should begin as a copy of Arm 1 wiring and firmware, but its automatic cycle should be developed as recorded waypoints. Do not begin with inverse kinematics. A waypoint cycle is easier to prove, safer to debug, and good enough for a small conveyor demo.

Phase	Goal	Done when
1. Bench copy	Build and calibrate the second arm exactly like Arm 1.	Every joint jogs without binding away from the conveyor.
2. Fixture mock	Place a dead/static part and mark pickup/drop zones.	Manual jog can pick and place with conveyor off.
3. Waypoint capture	Fill conveyor-waypoints-template.csv.	Each row has verified angles and notes.
4. Slow playback	Copy angles into WAYPOINTS and unlock demo.	Cycle runs at slow speed with no part.
5. Part trial	Run with part while conveyor is stopped or made safe.	Part transfers cleanly without contacting rails.
6. Conveyor integration	Add read-only conveyor state/interlock inputs later.	Arm only moves when the cell is safe and expected.

Do not trust hobby-servo position

The Servo library remembers the last commanded angle, not the actual physical position. If the arm is bumped, stalls, or starts from the wrong pose, the sketch does not know. Always home physically before servo power and before automatic playback.

9. Third Arm: Teleop and Training

The third arm can become a low-risk trainer. Keep it mechanically identical to the conveyor arm, but use it to teach poses, test gripper strategy, and practice operator workflows without tying up the real conveyor cell.

- Short term: use the same Uno serial controller and manually record angles from status output.
- Training workflow: jog to pose, type status, copy the six logical angles into a waypoint worksheet.
- Teleop upgrade: add joystick/pot inputs or move to an ESP32 controller later for wireless control.
- Do not let the teleop arm become the safety reference for the conveyor arm. The conveyor arm still needs its own clearance and limit verification.

10. Troubleshooting

Symptom	Most likely cause	Fix
Uno resets when servos move	Servo current is being pulled through USB/Uno or ground is poor.	Power servo red wires from external 5V supply; tie grounds; check supply voltage under load.
Servo jitters but does not move correctly	No common ground or weak signal connection.	Tie Arduino GND to servo ground bus; reseal signal wire; keep signal wires away from noisy power wiring.
Shoulder servos fight each other	Mirrored servo direction or horn position is wrong.	Cut servo power, remove load, flip SERVO_MIRRORED for that channel or re-center horns.
Servo growls at an endpoint	Software limit exceeds mechanical travel.	Reduce jointMin/jointMax immediately; do not hold a stalled servo.
Upload fails	D0/D1 are connected or serial monitor is open.	Keep D0/D1 unused; close Serial Monitor during upload.
Gripper crushes or drops part	Open/closed angles not tuned for the object.	Record separate gripper angles for empty, grip, and release in the waypoint sheet.

11. File Map

File	Use
docs/instructions/emre-kalem-robot-arm/wiring-map.csv	Servo-by-servo wiring table.
docs/instructions/emre-kalem-robot-arm/conveyor-waypoints-template.csv	Worksheet for the second arm's load/unload poses.
docs/instructions/emre-kalem-robot-arm/arduino/emre_kalem_arm_calibrate/	Calibration sketch.
docs/instructions/emre-kalem-robot-arm/arduino/emre_kalem_arm_uno_controller/	Main controller sketch.
tools/emre-kalem-arm-guide-pdf.py	This PDF generator.

12. Source Notes

The guide was built from these source facts. Treat the exact joint limits and waypoints as calibration values for your physical print, not universal constants.

Source	Used for
3DFinder mirror of MakerWorld model: https://3dfinder.io/model/makerworld/1134925-robotic-arm-with-servo-arduino and Emre Kalem assembly video: https://www.youtube.com/watch?v=CHV36hu9z3E	Emre Kalem model description, 6-axis project statement, and required parts list.
Emre original Arduino/software package linked from the assembly video.	Original pin map: D3 base, D4 Arm A1, D5 Arm A2, D6 Arm B, D9 Wrist A, D10 Wrist B, D11 gripper; Arm A2 mirrors as 180 - Arm A.
Related ESP32 firmware repo: https://github.com/peterz0310/robot-arm	Cross-check: 6 logical joints, 7 hobby servos, mirrored Arm A servo, external servo power, home-position/no-feedback warning.
Arduino Uno R3 docs/datasheet: https://docs.arduino.cc/hardware/uno-rev3 and https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf	Uno has 14 digital I/O, USB programming, ATmega328P, and standard board resources.
Arduino Servo library API/source: https://raw.githubusercontent.com/arduino-libraries/Servo/master/docs/api.md and https://raw.githubusercontent.com/arduino-libraries/Servo/master/src/Servo.h	attach(), write(), writeMicroseconds(), no physical feedback from read(), and endpoint overdrive high-current warning.
MG996R datasheet example: https://www.handsontec.com/dataspecs/motor_fan/MG996R.pdf	MG996R operating voltage/current expectations and high stall current.
MG90S datasheet example: https://www.tinytronics.nl/product_files/000263_Data%20Sheet%20of%20MG90S%20Analog%20Servo%20Motor.pdf	MG90S operating voltage and stall-current expectations.